

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA1516
---------------------	--	---------------------	---------

Course Outcome Covered

CO4: Analyze big data characteristics and challenges across domains including security and application aspects. (BL4)

Assessment Tool Weightage: 10%

Code of Conduct

I **AMARA SATWIKA - 192525204** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A.SATWIKA

Assessment Tasks — Reverse Engineering Task

Dave's Taxonomy: Articulation (Level 4) | Question Count: 5 | Total Marks: 30

For each task below, the described big data solution is studied, the probable build approach is inferred, underlying components are identified, and the likely design decisions are explained with supporting technical reasoning.

Q1. A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.

Probable Data Types

- Clickstream events (semi-structured JSON: user ID, timestamp, page/item viewed, device type).
- Search query logs (semi-structured text with metadata such as query string, filters applied, session ID).
- Purchase/transaction records (structured: order ID, item ID, price, payment status).
- Device/session metadata (structured: device type, OS, location) used to unify a user's activity across devices.

Inferred Ingestion Flow

Given that events arrive continuously from many devices, the system was most likely built with an event-driven ingestion layer such as Apache Kafka or a similar message queue, where each click, search, and purchase is published as an event. A stream processor (e.g., Spark Streaming or Flink) probably consumes these events, tags them with a unified user/session ID, and writes raw events into a distributed data lake (HDFS/S3) for long-term storage, while also forwarding a filtered subset to a fast, low-latency store for near-real-time personalization.

Probable Analytics Stages

- Stage 1 – Raw ingestion & storage: Events land in the data lake in their raw form, partitioned by date/user for efficient batch processing later.
- Stage 2 – Feature engineering: Batch jobs (Spark) aggregate click/search/purchase history into user and item feature vectors (e.g., recency, frequency of interaction).
- Stage 3 – Model training: Collaborative filtering or matrix-factorization models (or deep learning embeddings) are trained periodically offline on the aggregated features.
- Stage 4 – Real-time scoring/serving: Trained model outputs are pushed to a fast key-value store (e.g., Redis/Cassandra) so recommendations can be served with low latency as the user browses.

Design decision inference: Separating slow offline model training from fast online serving is a strong indicator that the designers prioritized both analytical depth (batch) and responsiveness (real-time serving) — a classic lambda/kappa-architecture pattern used in large-scale recommendation systems.

Q2. A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.

Inferred Backbone

- **Ingestion:** GPS pings from thousands of driver and rider apps arrive continuously, which strongly suggests a high-throughput streaming ingestion layer (Apache Kafka) capable of handling millions of location events per minute.
- **Stream processing:** To render a live heatmap, location events almost certainly pass through a real-time stream processor (Spark Streaming, Flink, or Kafka Streams) that aggregates GPS points into geographic grid cells (e.g., using geohashing) and computes rolling counts of active rides/demand per cell every few seconds.
- **Distributed storage:** Historical trip and location data — needed for demand forecasting — is likely stored in a distributed file system (HDFS or cloud object storage) partitioned by time and region, enabling efficient batch analysis of long-term demand patterns.
- **Serving layer:** The live heatmap itself is probably served from a fast in-memory or time-series store (e.g., Redis or a specialized geospatial database) that is updated continuously by the stream processor, since a data lake alone would be too slow for sub-second map refreshes.
- **Forecasting layer:** Demand forecasts likely come from a separate batch/ML pipeline (Spark MLlib or a time-series model) trained periodically on historical trip data stored in the data lake, with outputs refreshed at intervals (e.g., every 15 minutes) rather than continuously.

Design Decision Inference

The clear separation between a low-latency serving path (for the live heatmap) and a batch analytics path (for demand forecasting) indicates a deliberate architectural choice to balance the Velocity requirement of real-time visualization against the Volume/complexity requirement of accurate long-term forecasting — a design that would be very difficult to achieve with a single monolithic database.

Q3. A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.

Probable Streaming Mechanism

Because fraud must be flagged within seconds of a transaction, the system almost certainly ingests transactions through a streaming platform such as Apache Kafka, with a stream-processing engine (Spark Streaming or Flink) applying a pre-trained fraud-scoring model to each transaction as it arrives, rather than waiting for batch processing.

Probable Security Mechanisms

- Tokenization/masking of card numbers and PII at ingestion, so raw sensitive data is never exposed to the analytics layer — inferred from the strict privacy requirements typical of financial systems.
- Encryption in transit (TLS) between payment terminals, the streaming layer, and storage, and encryption at rest for any persisted transaction data.
- Role-based access control restricting dashboard visibility to fraud-analysis and compliance teams only, with all access logged for audit purposes.
- Network segmentation/isolation of the fraud-detection pipeline from general-purpose analytics systems, reducing the attack surface for sensitive financial data.

Probable Alerting Mechanism

The stream processor likely applies a trained anomaly-detection or classification model (e.g., logistic regression, gradient boosting, or a rules engine combined with ML scoring) to each transaction, and any transaction scoring above a risk threshold is pushed to an alerting service (e.g., via a message queue to a notification system) that immediately updates the dashboard and may trigger an automatic hold or SMS/app verification step.

Design decision inference: The near-real-time requirement rules out a purely batch architecture; the presence of both scoring (velocity) and strict privacy controls (veracity/security) suggests the designers deliberately traded some model complexity for speed, favoring lightweight, low-latency scoring models over heavier offline-only models.

Q4. A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.

Inferred Data Management Approach

- **Unstructured data (videos):** Almost certainly stored as binary objects in a distributed object store or DFS (e.g., HDFS or a cloud blob store), since videos are large and not suited to relational storage; a content-delivery network (CDN) likely caches videos for fast global playback.
- **Semi-structured data (comments, hashtags):** Likely stored in a NoSQL document or wide-column store (e.g., Cassandra or MongoDB) that can flexibly handle nested comment threads and variable hashtag counts per post without a rigid schema.
- **Structured data (engagement metrics — likes, shares, view counts):** Probably stored in a fast key-value or columnar store, updated in near real time as users interact, and periodically aggregated into a data warehouse for reporting/analytics.
- **Linking mechanism:** A common post ID / content ID almost certainly ties the video object, its comments, hashtags, and engagement counters together, with a metadata index (not the raw video itself) used whenever the system needs to search, filter, or rank content.

Design Decision Inference

Storing each data type in the storage system best suited to it (object store for video, NoSQL for comments/hashtags, fast counters for engagement) — rather than forcing everything into one database — indicates a polyglot persistence design. This is a strong architectural signal that the platform prioritized scalability and access-pattern optimization (fast video streaming, flexible comment structure, real-time counters) over the simplicity of a single unified database.

Q5. A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

Probable Integration Tools

Since machine sensors across many plants generate continuous telemetry (temperature, vibration, error codes) alongside periodic maintenance event logs, the system likely uses an edge/gateway layer at each plant to collect and forward sensor data, with a message broker such as Apache Kafka or MQTT (common in IoT) aggregating streams from all plants into a central pipeline. A data-integration tool such as Apache NiFi likely handles routing, format standardization, and reliable delivery given the mix of protocols different plants and machine vendors may use.

Probable Storage Strategy

- Raw high-frequency sensor logs are likely stored in a distributed file system or a time-series database (e.g., HDFS combined with InfluxDB/OpenTSDB), since time-stamped machine metrics fit time-series storage patterns better than relational tables.
- Maintenance event records (more structured — technician, date, action taken) are probably stored separately in a relational or structured store, then linked to sensor data via machine ID and timestamp for joint analysis.
- Data is likely partitioned by plant and date to support efficient querying of a specific plant's history without scanning the entire multi-plant dataset.

Probable Analytics Path

- Batch analytics (Spark) on historical logs to identify long-term wear patterns and correlate sensor anomalies with past failures.
- Predictive maintenance models trained on this historical data to forecast when a machine is likely to fail, allowing plants to schedule maintenance proactively rather than reactively.
- Real-time/near-real-time stream monitoring to raise immediate alerts when a sensor reading crosses a critical threshold (e.g., overheating), independent of the slower batch/ML pipeline.

Design decision inference: The likely dual-path design — fast threshold-based alerting for immediate safety issues, plus slower ML-based predictive maintenance for long-term planning — reflects a deliberate trade-off between the Velocity needs of operational safety and the Value extracted from deeper historical analysis, a pattern very common in industrial IoT (IIoT) architectures.
